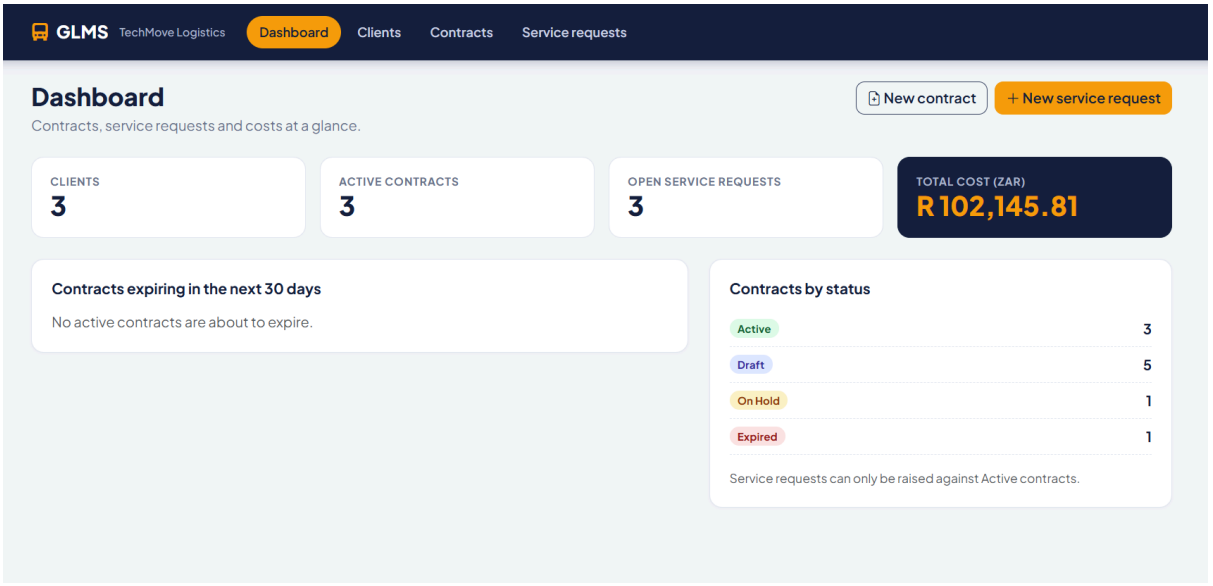
Mordernaisation

Figure 1. GLMS dashboard — Overview of clients, active contracts, open service requests, total cost in ZAR, and contract status counts.



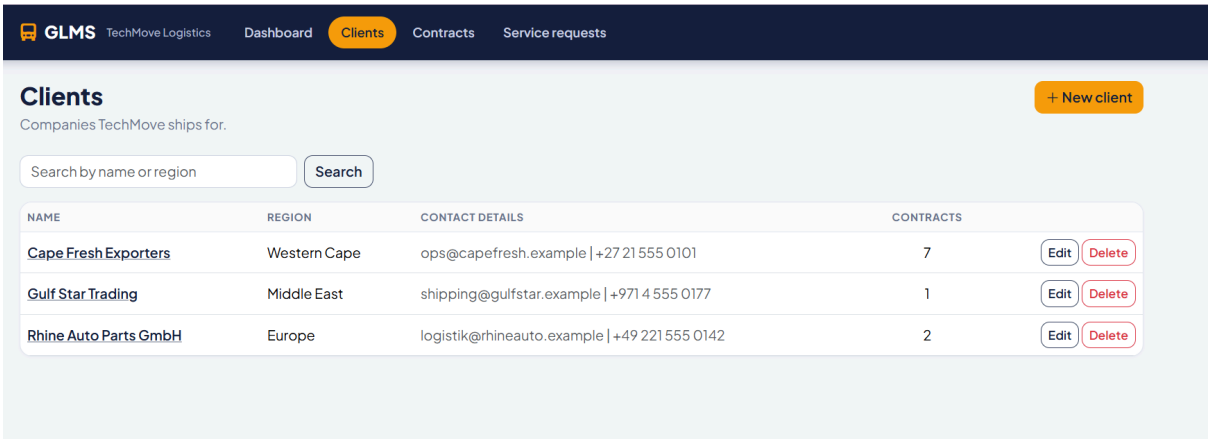


Figure 2. Client directory — Displays registered clients with their regions, contact details, contract totals, and edit/delete actions.

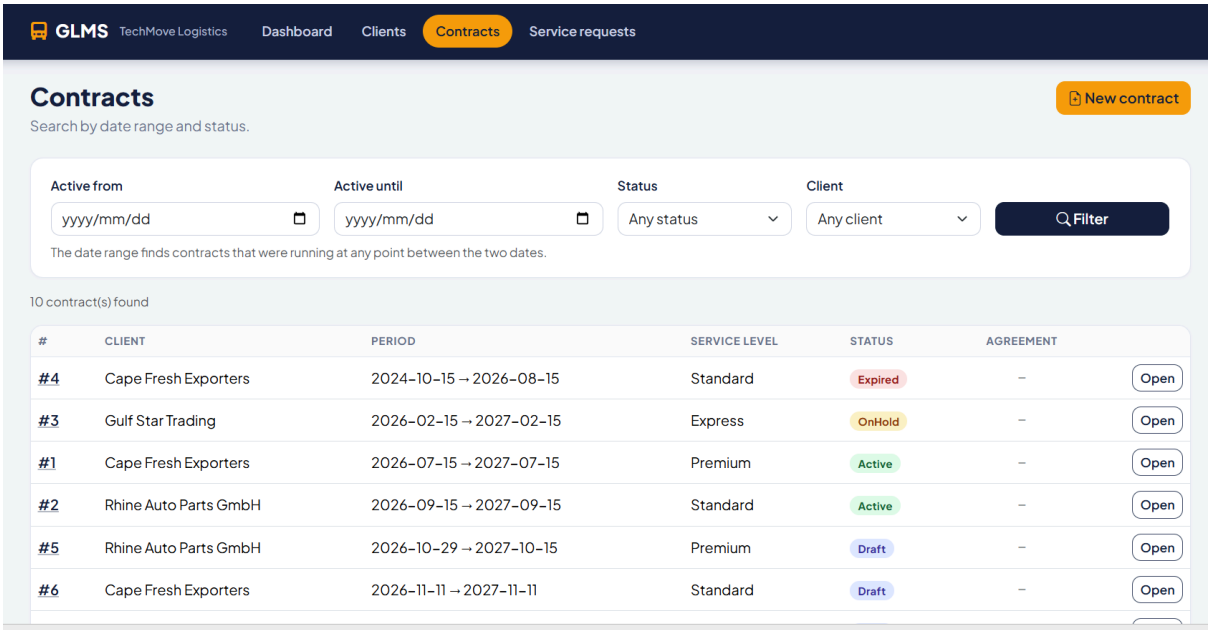


Figure 3. Contract register — Lists contracts with client, date range, service level, status, agreement indicator, and filtering controls.

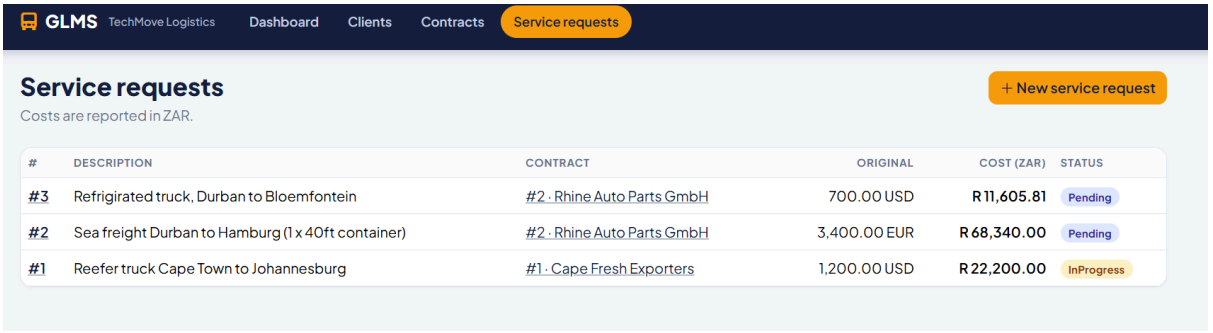


Figure 4. Service request register — Shows service requests linked to contracts, including descriptions, original costs, converted ZAR amounts, and statuses.

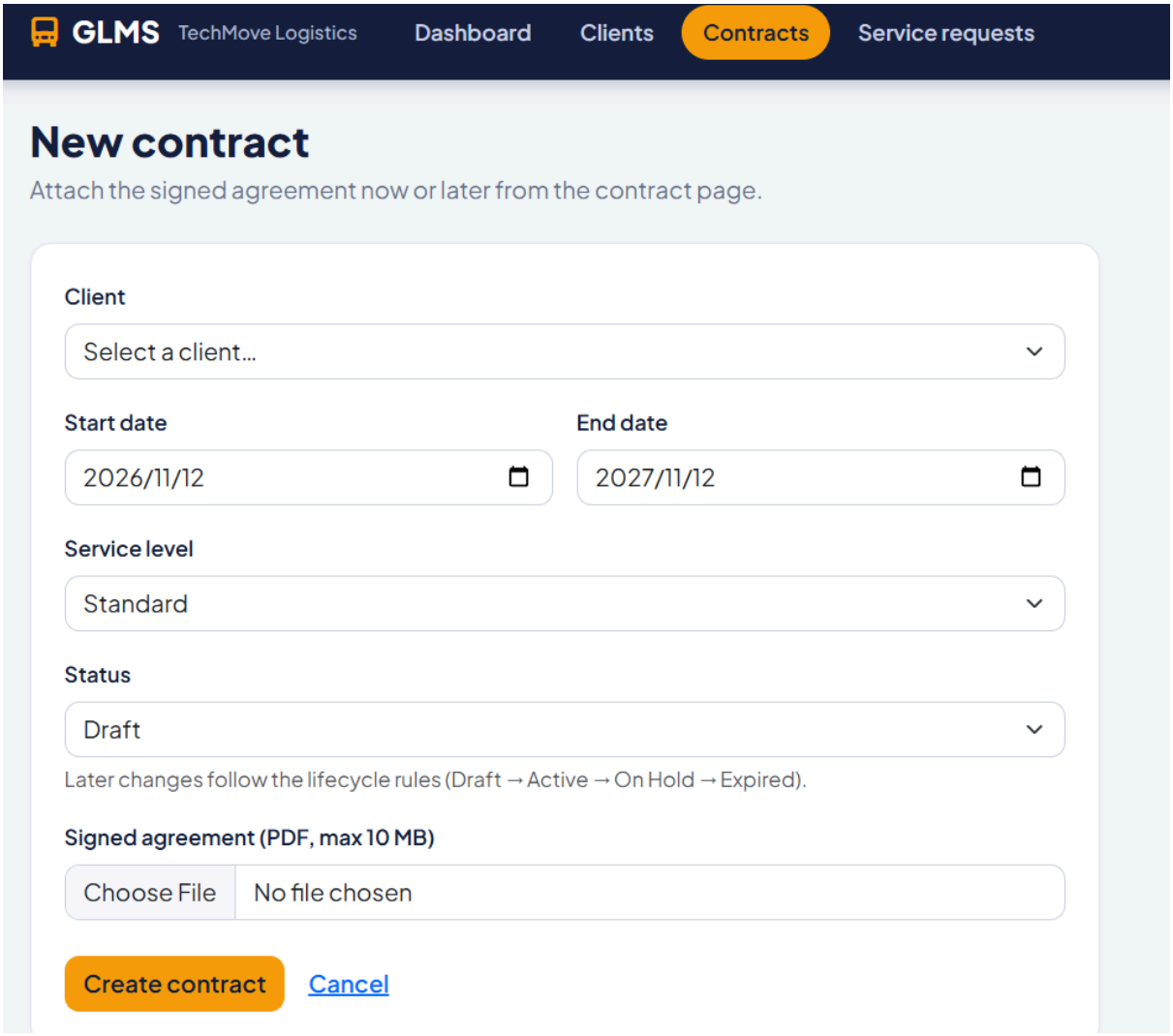


Figure 5. New contract form — Captures the client, contract dates, service level, initial status, and optional signed PDF agreement.

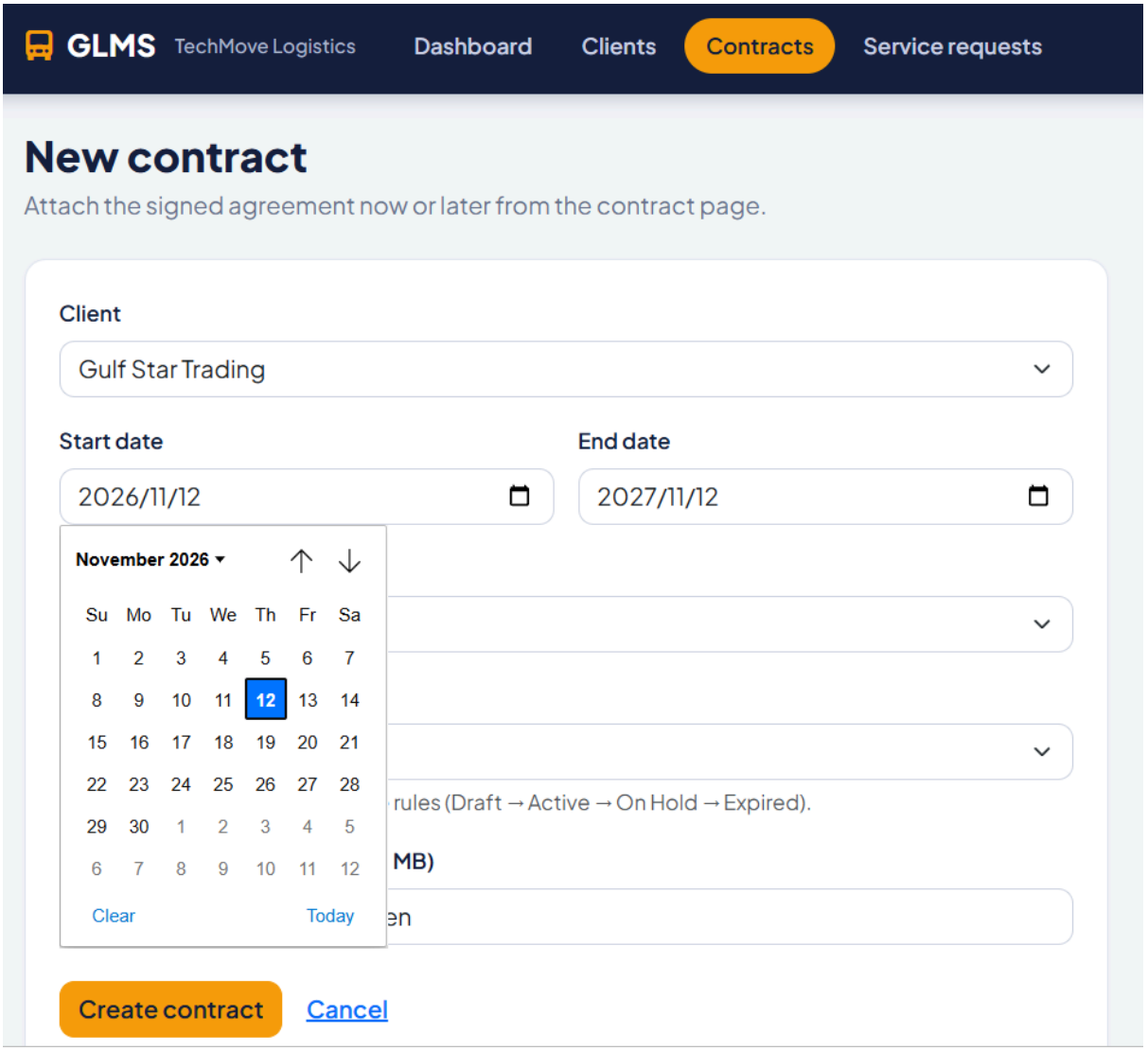


Figure 6. Contract start-date picker — Calendar control used to select a contract start date.

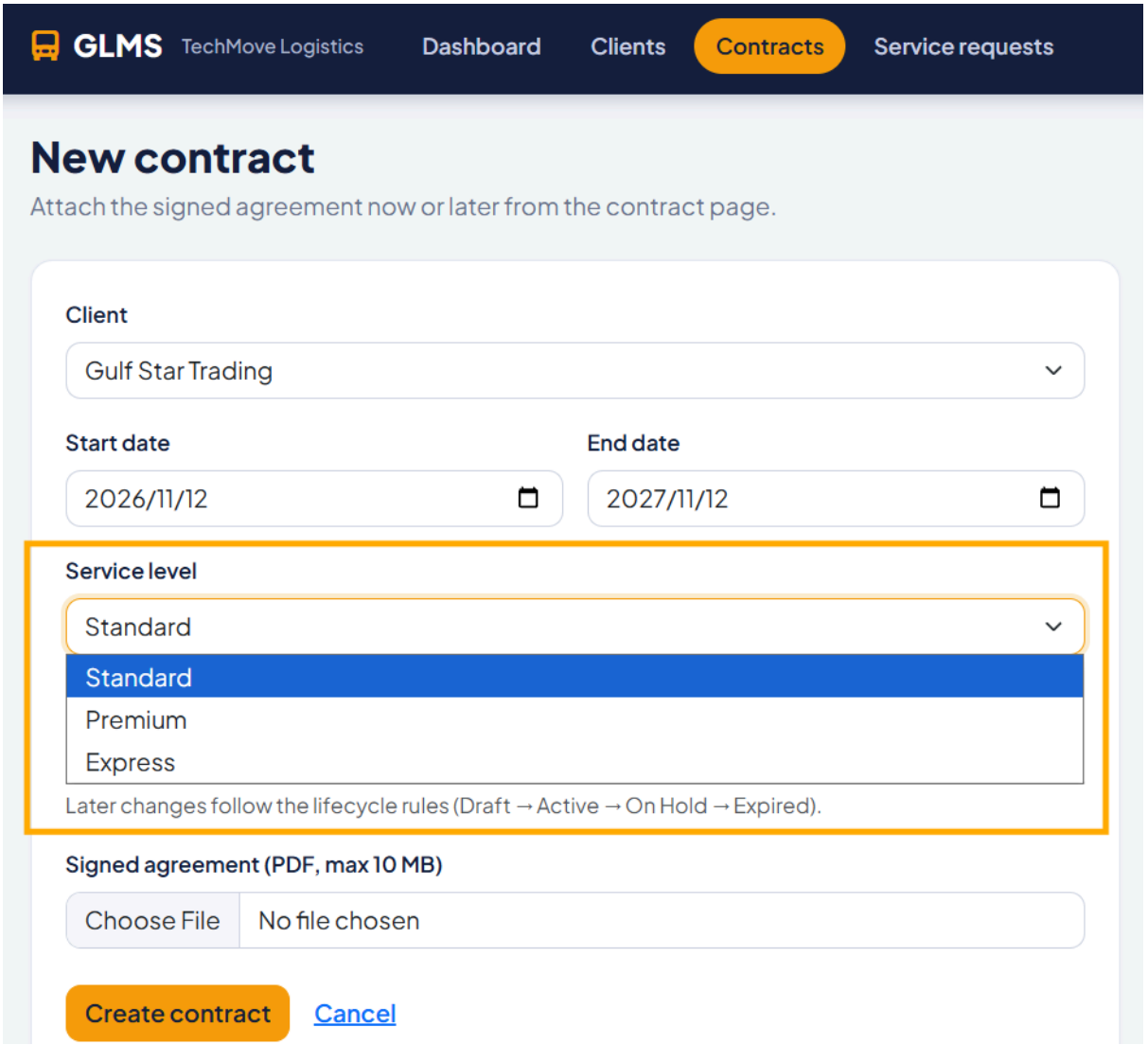


Figure 7. Service level selection — Available service levels shown in the dropdown: Standard, Premium, and Express.

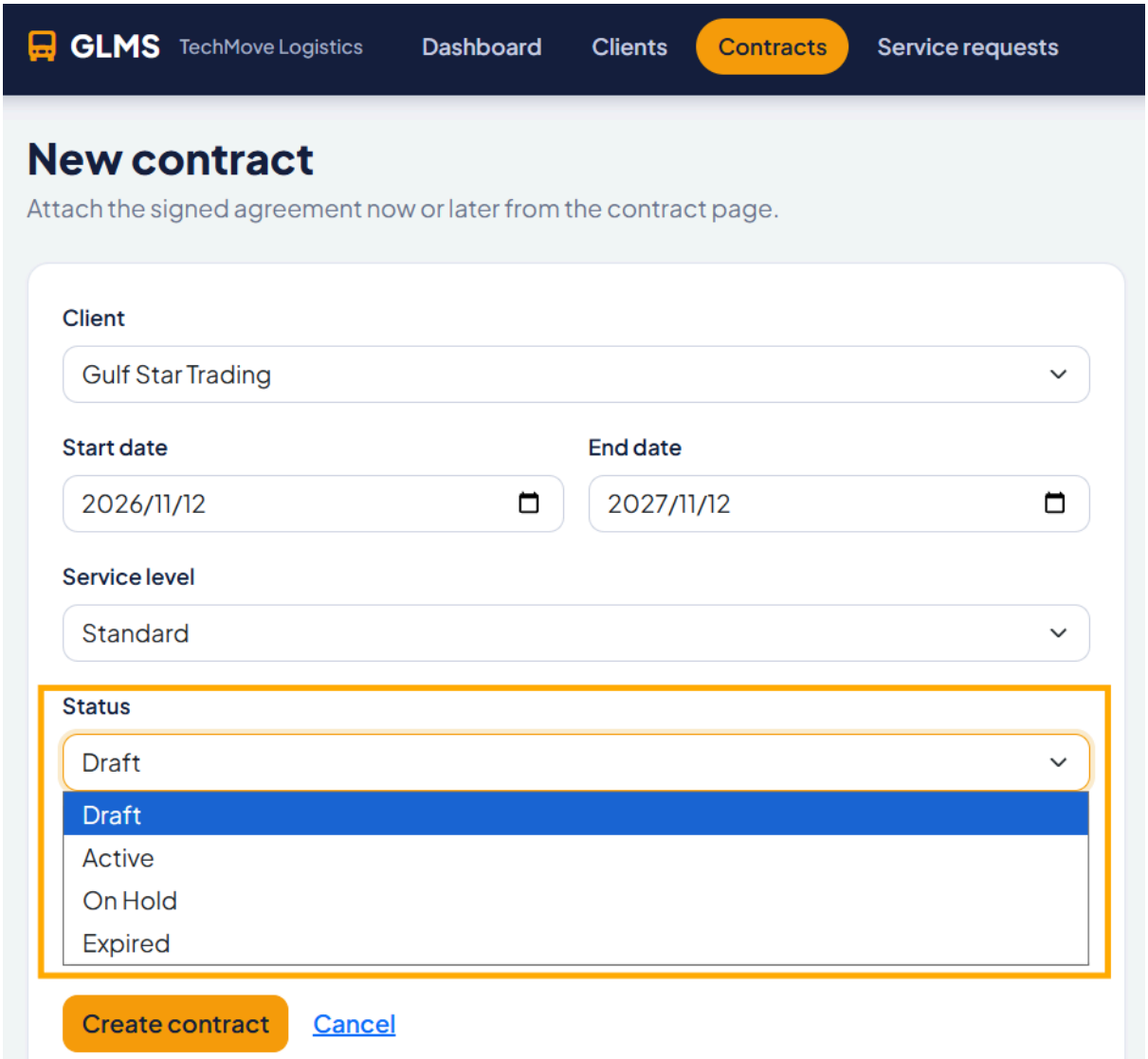


Figure 8. Contract status selection — Dropdown displays Draft, Active, On Hold, and Expired status options.

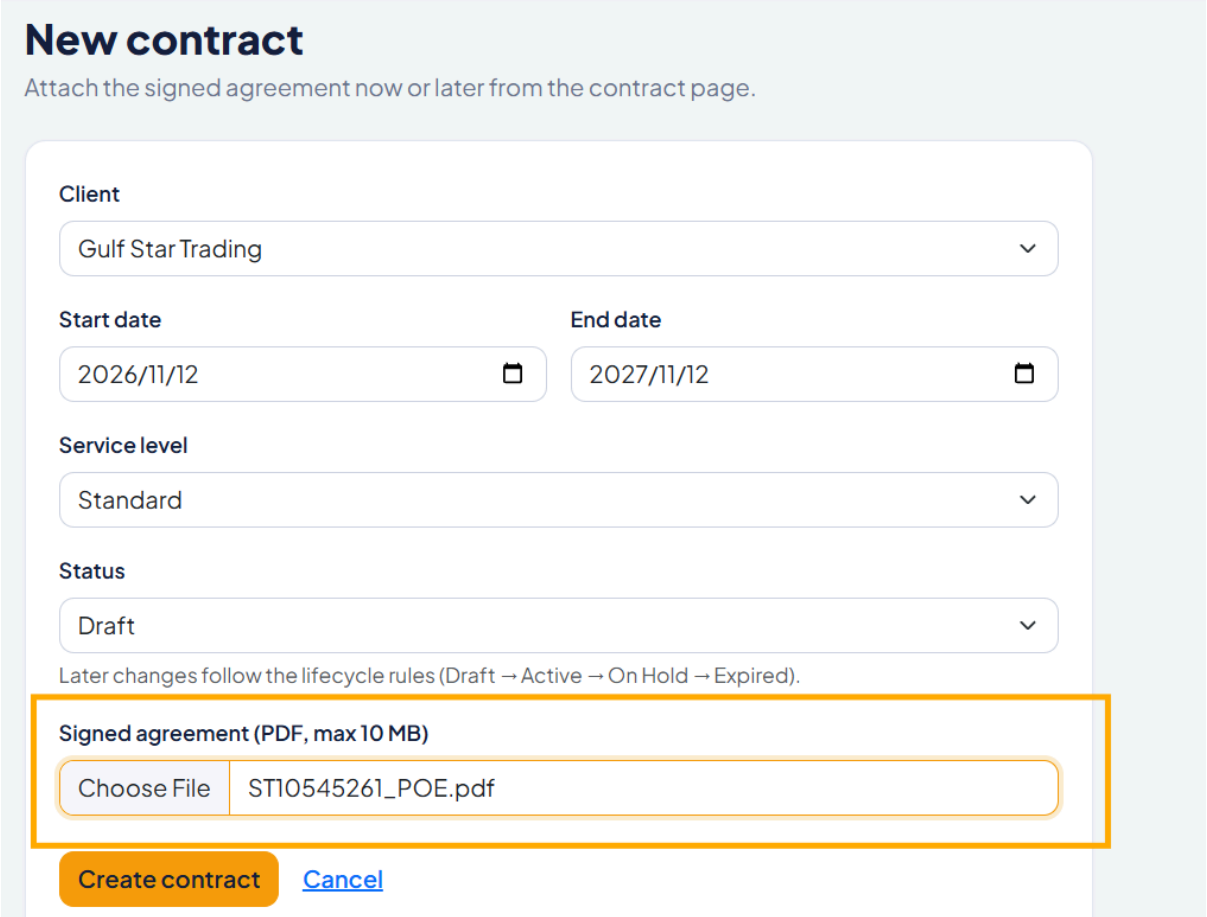


Figure 9. Signed agreement attachment — PDF file selected for upload when creating a contract; the form indicates a 10 MB limit.

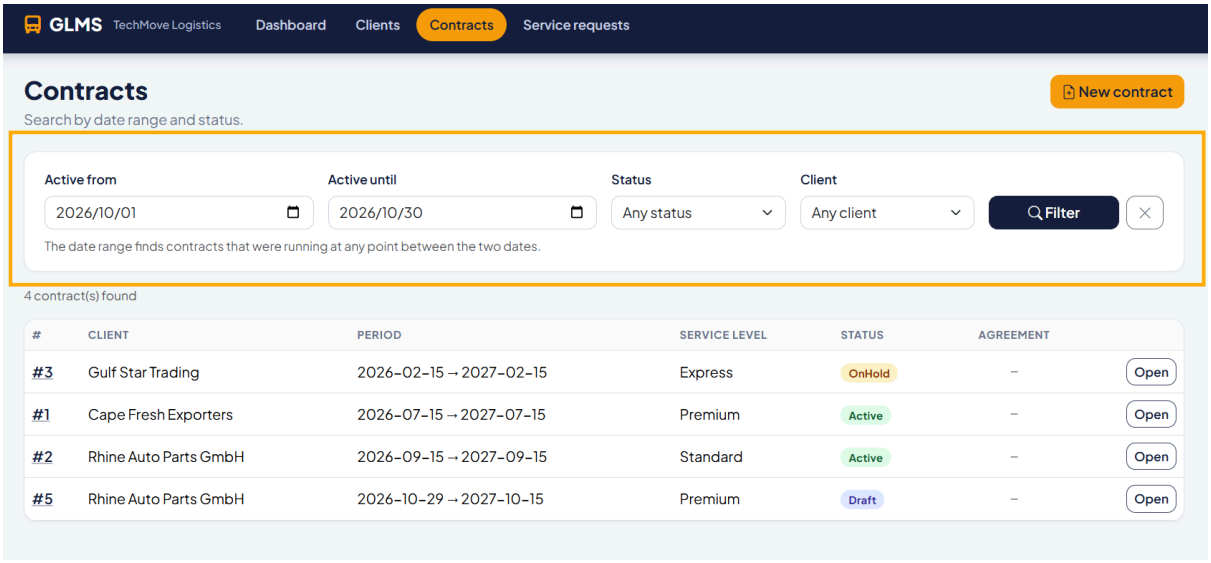


Figure 10. Contract date filtering — The contract register can be narrowed using the Active from and Active until date fields.

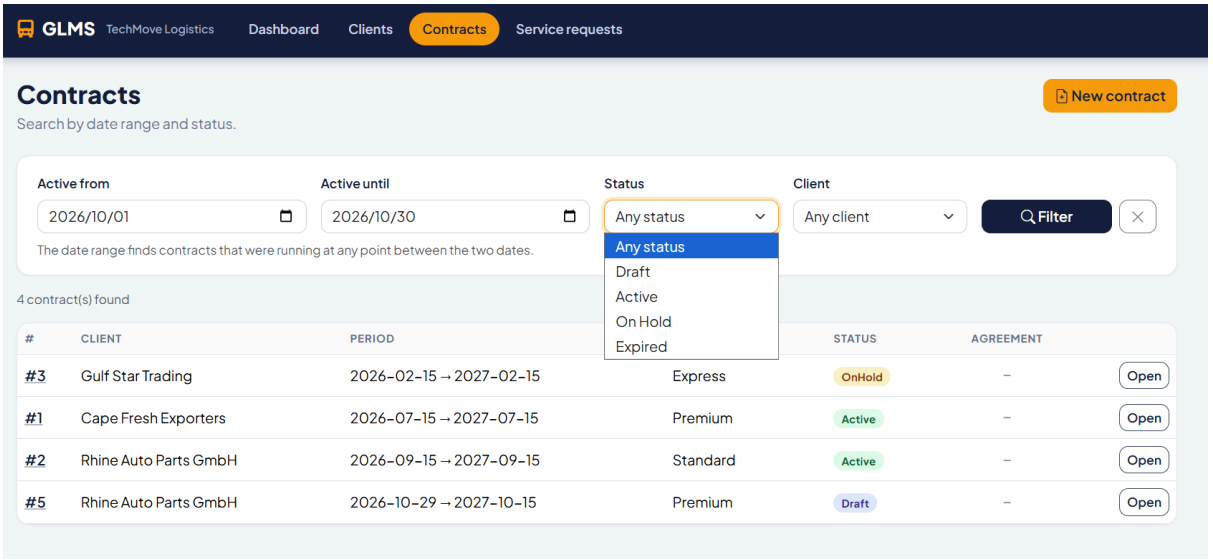


Figure 11. Contract status filter — Status dropdown allows the contract list to be filtered by lifecycle state.

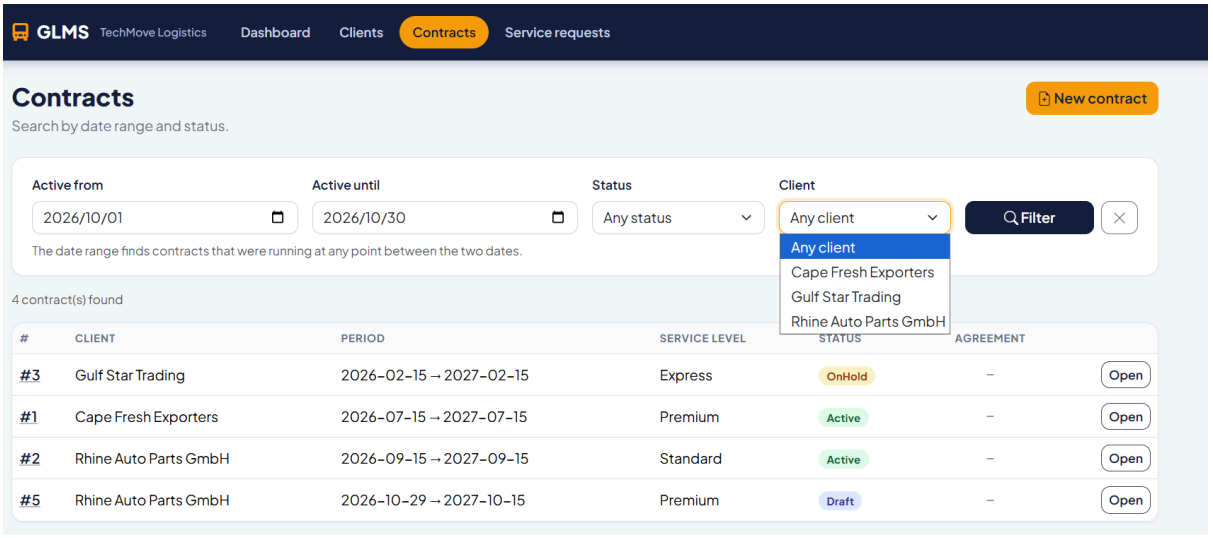


Figure 12. Contract client filter — Client dropdown restricts results to contracts associated with a selected customer.

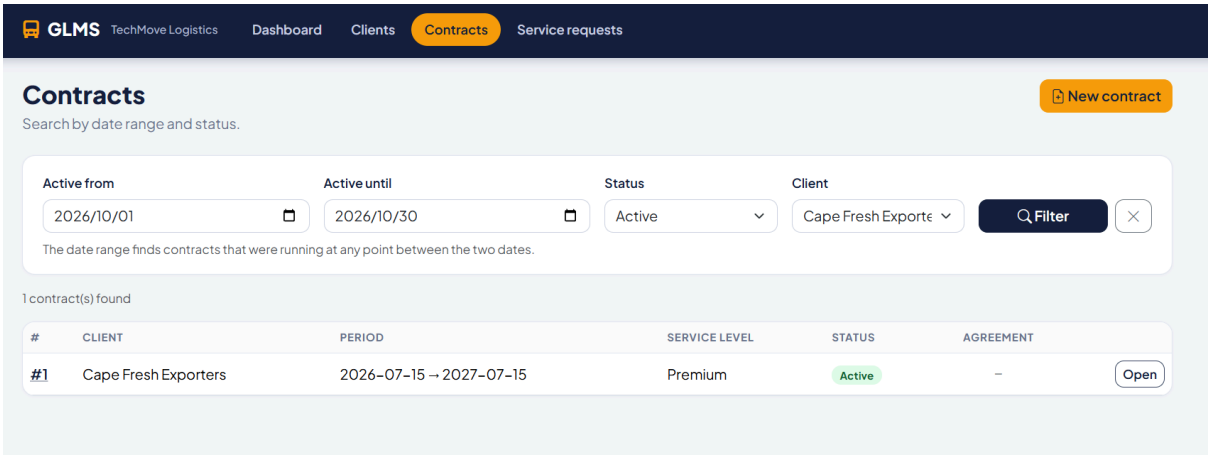


Figure 13. Combined contract filters — Example of a filtered contract list using a date range, Active status, and a specific client.

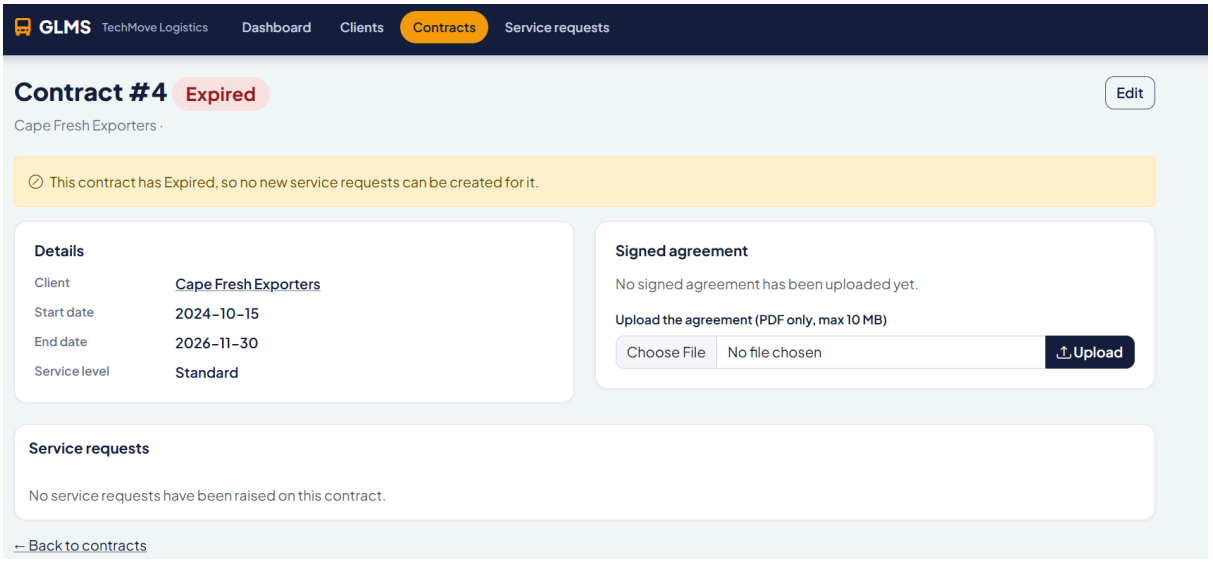


Figure 14. Expired contract details — Contract detail view shows the Expired status, client, dates, service level, and signed-agreement upload section.

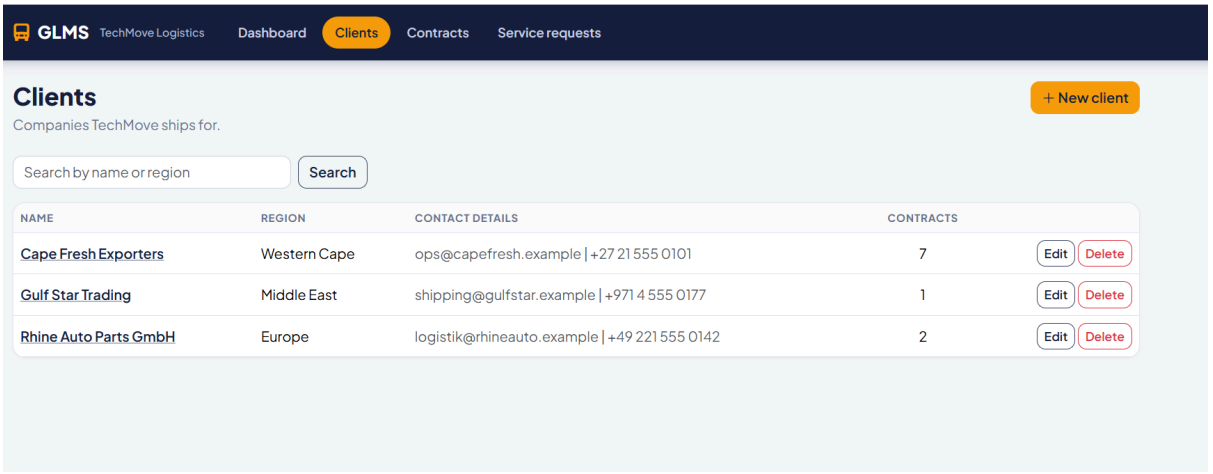


Figure 15. Client list and search — Client management page provides a searchable directory and actions for each client.

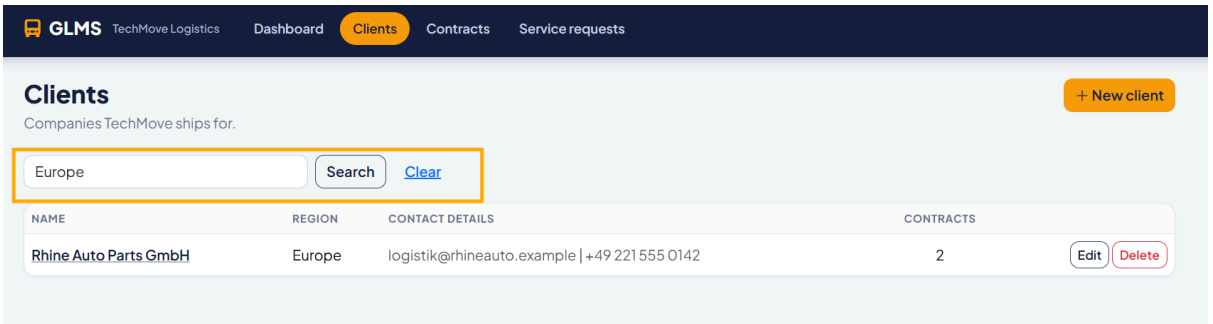


Figure 16. Client search results — Filtering the client list by Europe returns the matching regional client.

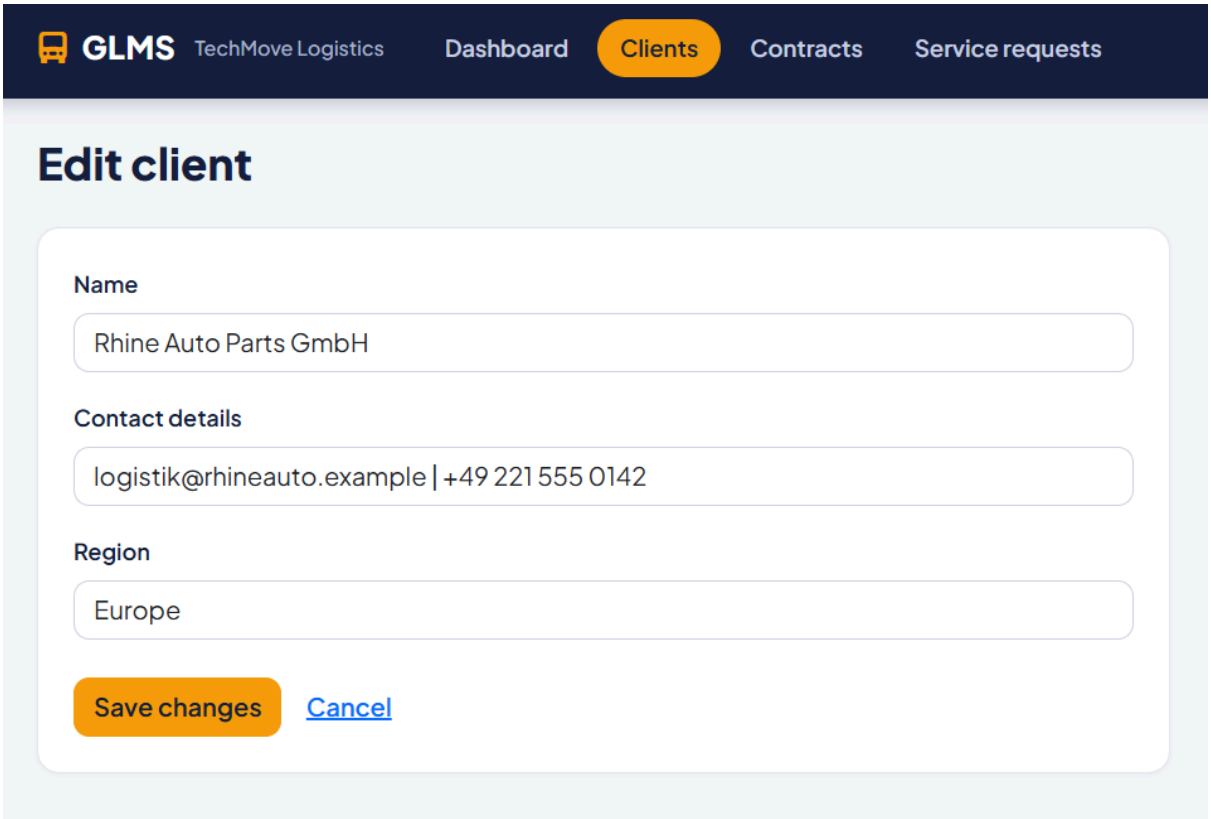


Figure 17. Edit client form — Allows updates to the client name, contact information, and region.

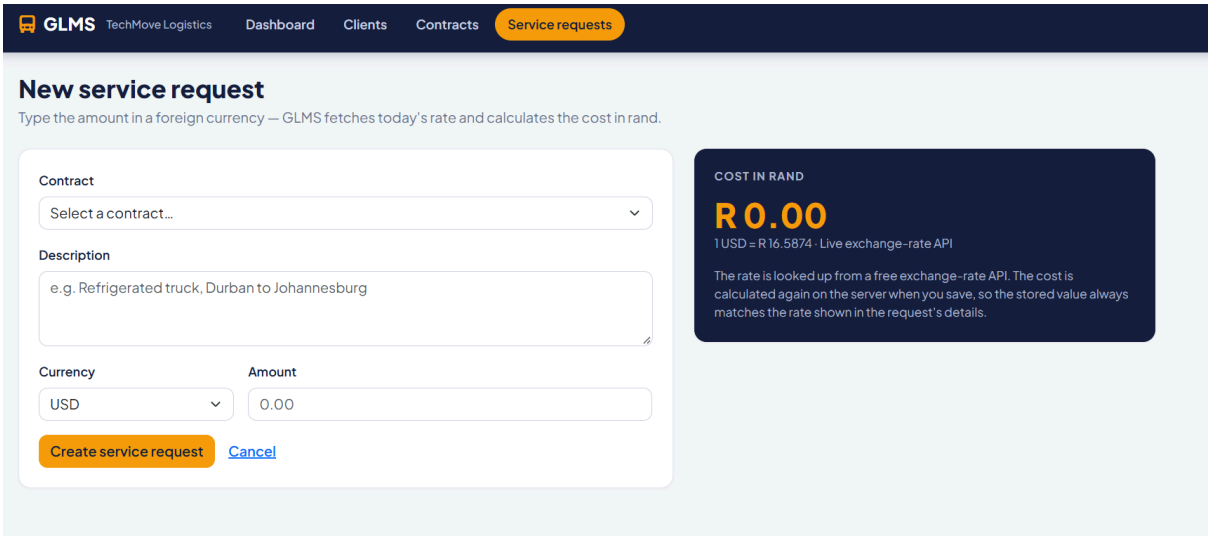


Figure 18. New service request form — Captures the associated contract, request description, currency, and amount with a ZAR cost preview.

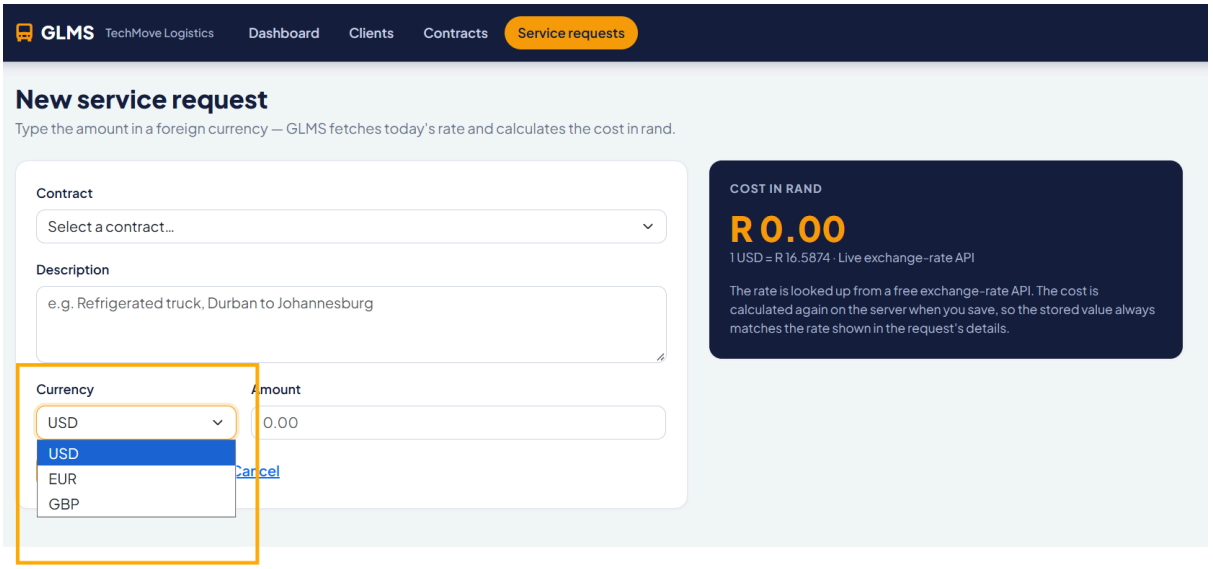


Figure 19. Service request currency selection — Currency dropdown offers USD, EUR, and GBP for entering the original service request amount.

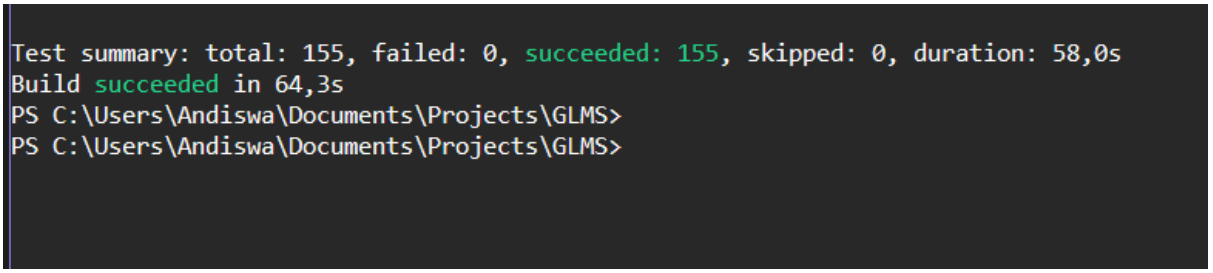
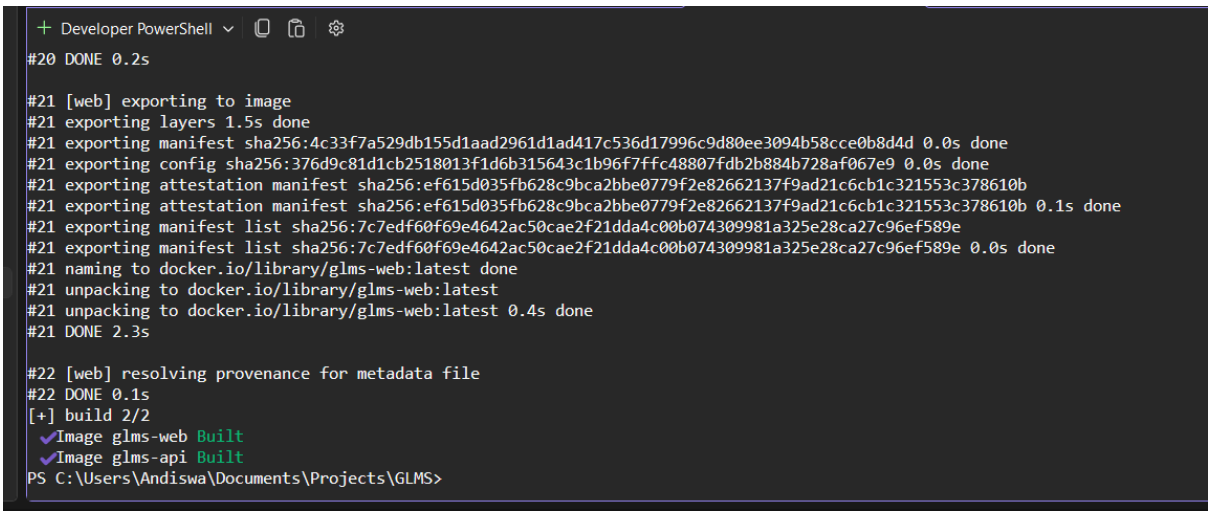


Figure 20: GLMS Automated Test Execution Results

The GLMS automated test suite was executed using the .NET CLI and xUnit testing framework. All 155 tests passed successfully, with zero failures and zero skipped tests. The results demonstrate successful execution of the implemented unit and API integration tests.



Docker

Figure: Successful Docker Compose Build of GLMS Application Images

The screenshot demonstrates the successful containerisation of the Global Logistics Management System (GLMS) using Docker Compose. Both the GLMS Web API (`glms-api`) and MVC frontend (`glms-web`) images were built successfully, with no build errors. This confirms that the application's Dockerfiles and build configuration are functioning correctly.

POE Section: Containerisation and Deployment – Docker Build Verification.

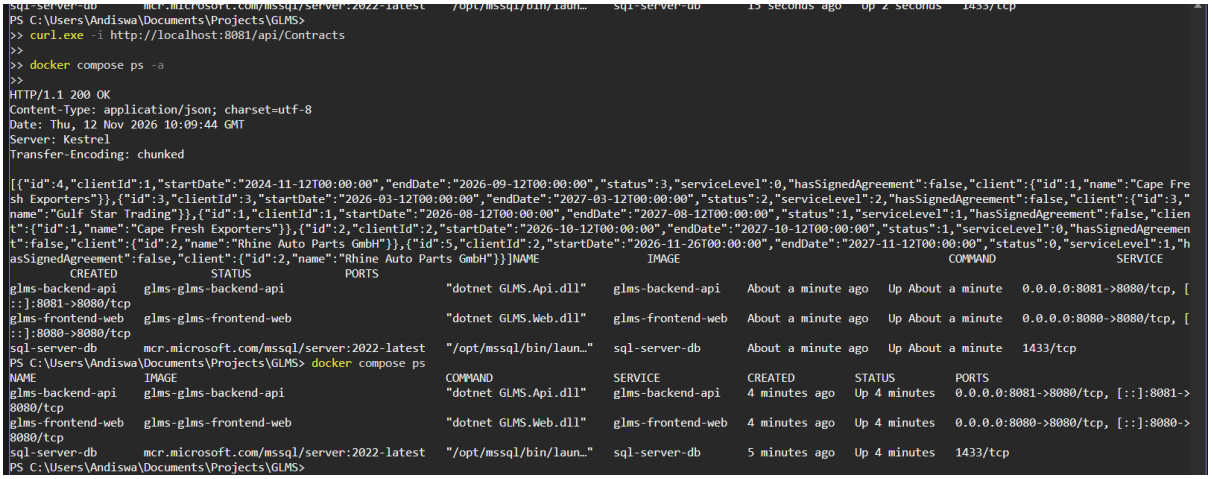


Figure 1: GLMS Three-Container Deployment

The screenshot demonstrates the deployment of the GLMS application using Docker Compose. All three required containers—SQL Server database, ASP.NET Core Web API and MVC frontend—are running, with ports 8080 and 8081 exposed for the frontend and API respectively.

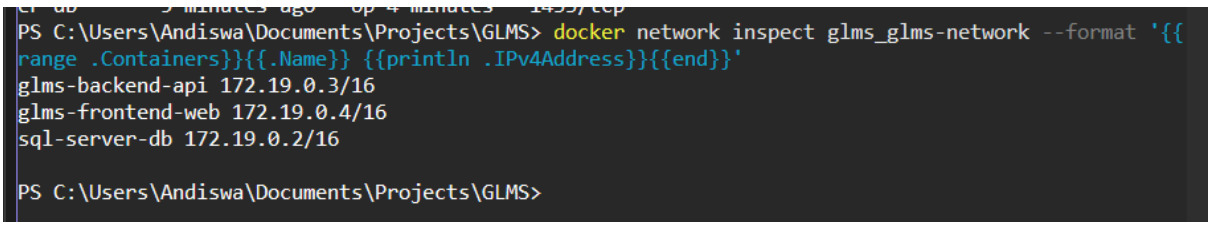


Figure 2: GLMS Internal Docker Network Configuration

The screenshot demonstrates the internal Docker network connecting the SQL Server database, Web API and MVC frontend containers. This network enables communication between the services using Docker's internal networking infrastructure.

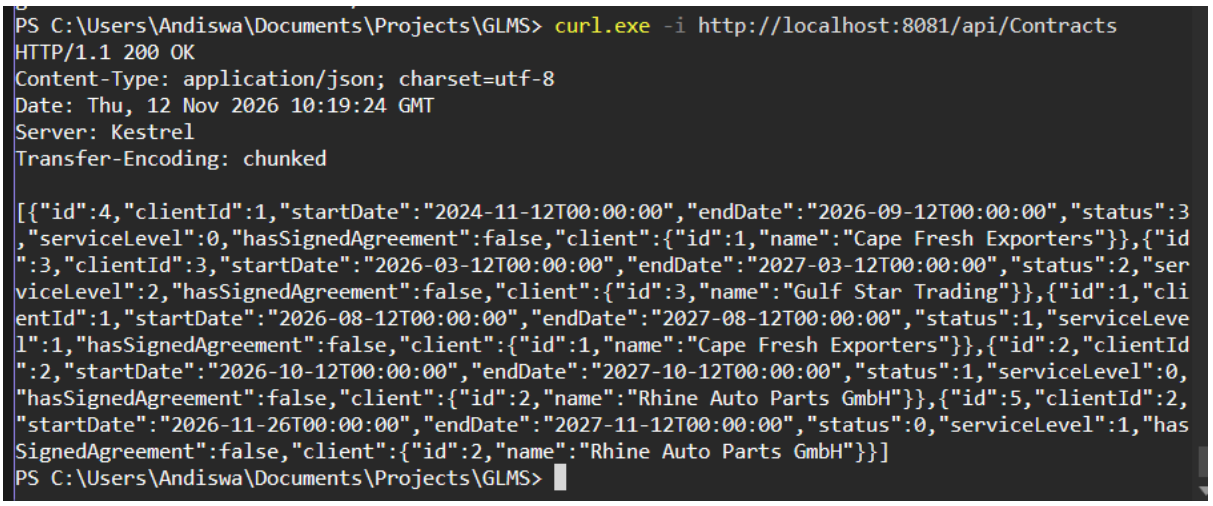


Figure 3: Successful API and SQL Server Integration

The screenshot shows the GLMS Contracts API successfully returning JSON contract records from the containerised SQL Server database. The response demonstrates that the backend service can communicate with the database and retrieve stored information.

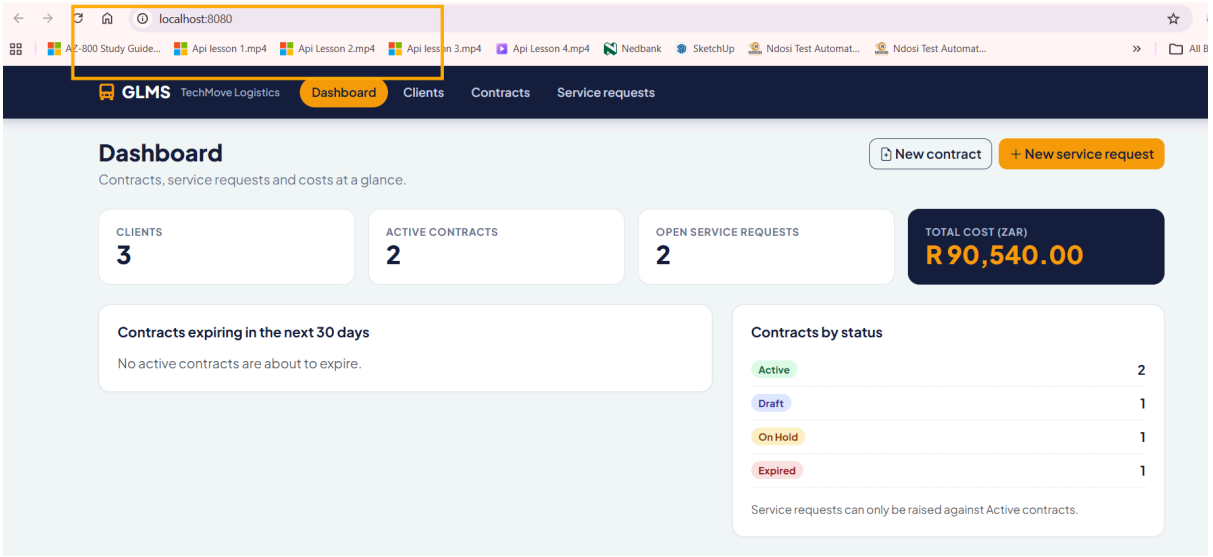


Figure 4: GLMS MVC Frontend Running in Docker

The screenshot demonstrates the ASP.NET Core MVC frontend being accessed through the Docker-published port 8080. The dashboard displays the application's interface and confirms successful browser access to the containerised frontend.

```
PS C:\Users\Andiswa\Documents\Projects\GLMS> Get-Content .\docker-compose.yml

services:

  sql-server-db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: sql-server-db
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_SA_PASSWORD: "${SA_PASSWORD}"
      MSSQL_PID: "Developer"
      MSSQL_MEMORY_LIMIT_MB: "2048"
    volumes:
      - sql_data:/var/opt/mssql
    networks:
      - glms-network
    restart: unless-stopped

  glms-backend-api:
    build:
      context: .
      dockerfile: GLMS.Api/Dockerfile
    container_name: glms-backend-api
    ports:
      - "8081:8080"
    environment:
      ASPNETCORE_ENVIRONMENT: Development
      ASPNETCORE_HTTP_PORTS: "8080"
      ConnectionStrings__GlmsDb: "Server=sql-server-db,1433;Database=GlmsDb;User Id=sa;Password=${SA_PASSWORD};TrustServerCertificate=True;"
    depends_on:
      - sql-server-db
    volumes:
      - agreements:/app/App_Data/SignedAgreements
    networks:
      - glms-network

  glms-frontend-web:
    build:
      context: .
      dockerfile: GLMS.Web/Dockerfile
    container_name: glms-frontend-web
    ports:
      - "8080:8080"
    environment:
      ASPNETCORE_ENVIRONMENT: Development
      ASPNETCORE_HTTP_PORTS: "8080"
      ApiBaseUrl: "http://glms-backend-api:8080/"
      ConnectionStrings__GlmsDb: "Server=sql-server-db,1433;Database=GlmsDb;User Id=sa;Password=${SA_PASSWORD};TrustServerCertificate=True;"
    depends_on:
      - glms-backend-api
    networks:
      - glms-network

volumes:
  sql_data:
  agreements:

networks:
  glms-network:
    driver: bridge

PS C:\Users\Andiswa\Documents\Projects\GLMS>
```

Figure 5: Docker Compose Configuration for the GLMS Architecture

The screenshot illustrates the Docker Compose configuration defining three services: SQL Server, the GLMS Web API and the MVC frontend. The configuration includes container networking, database persistence and port mappings.

Containerization

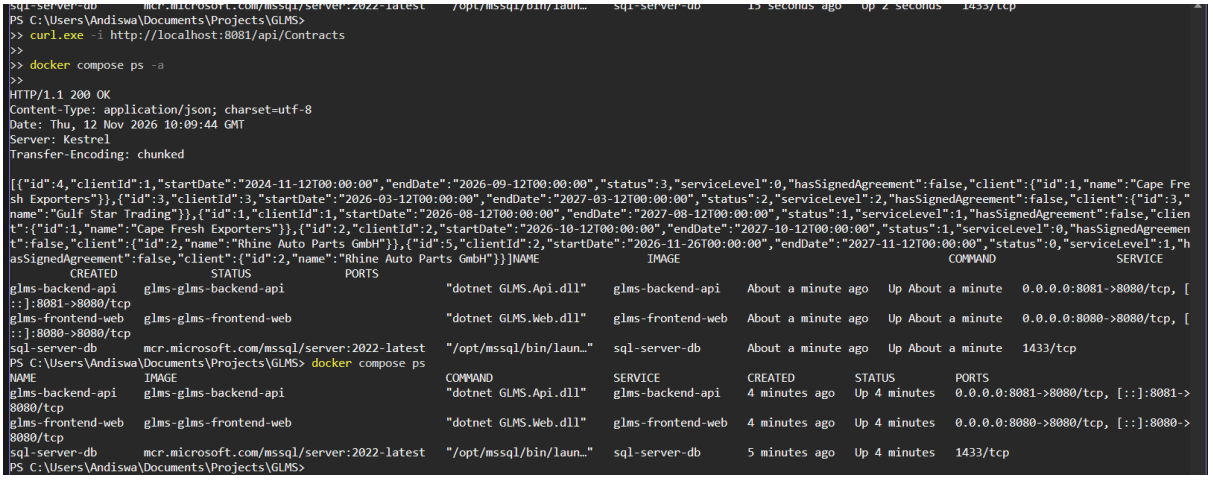


Figure 1: GLMS Three-Container Deployment

The screenshot demonstrates the deployment of the GLMS application using Docker Compose. All three required containers—SQL Server database, ASP.NET Core Web API and MVC frontend—are running, with ports 8080 and 8081 exposed for the frontend and API respectively.

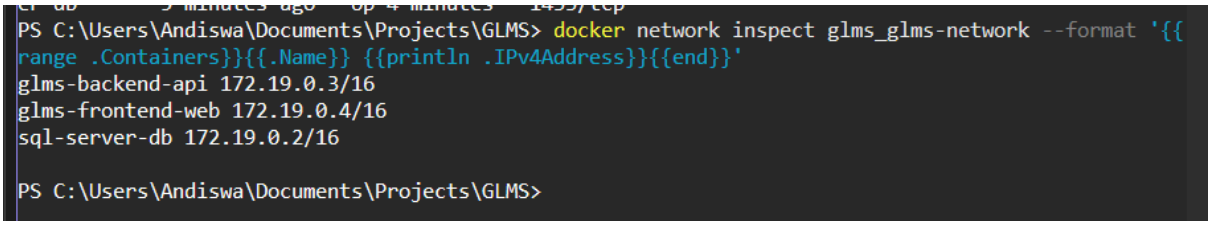


Figure 2: GLMS Internal Docker Network Configuration

The screenshot demonstrates the internal Docker network connecting the SQL Server database, Web API and MVC frontend containers. This network enables communication between the services using Docker's internal networking infrastructure.

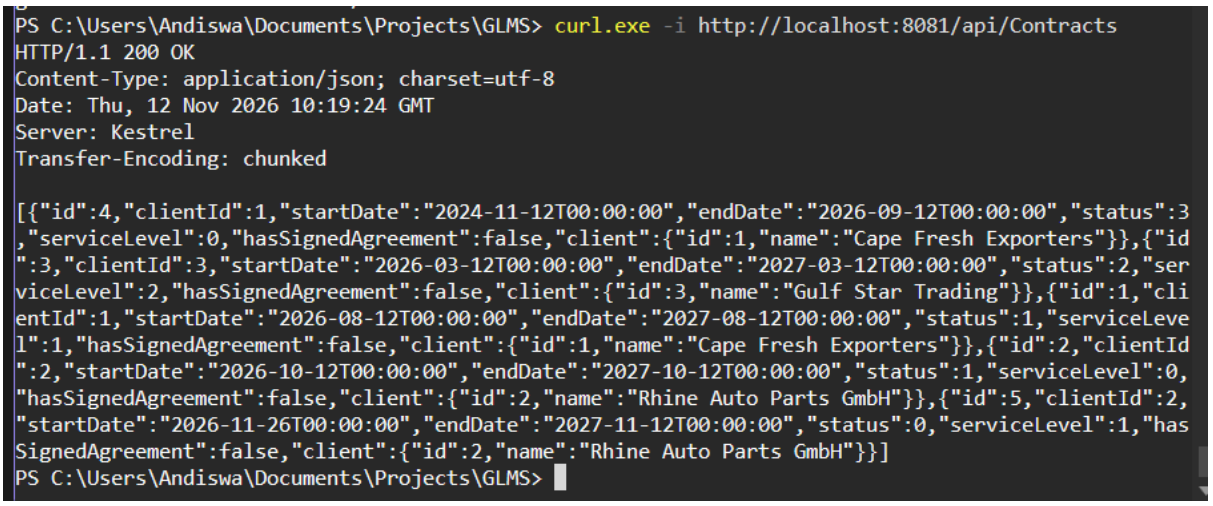


Figure 3: Successful API and SQL Server Integration

The screenshot shows the GLMS Contracts API successfully returning JSON contract records from the containerised SQL Server database. The response demonstrates that the backend service can communicate with the database and retrieve stored information.

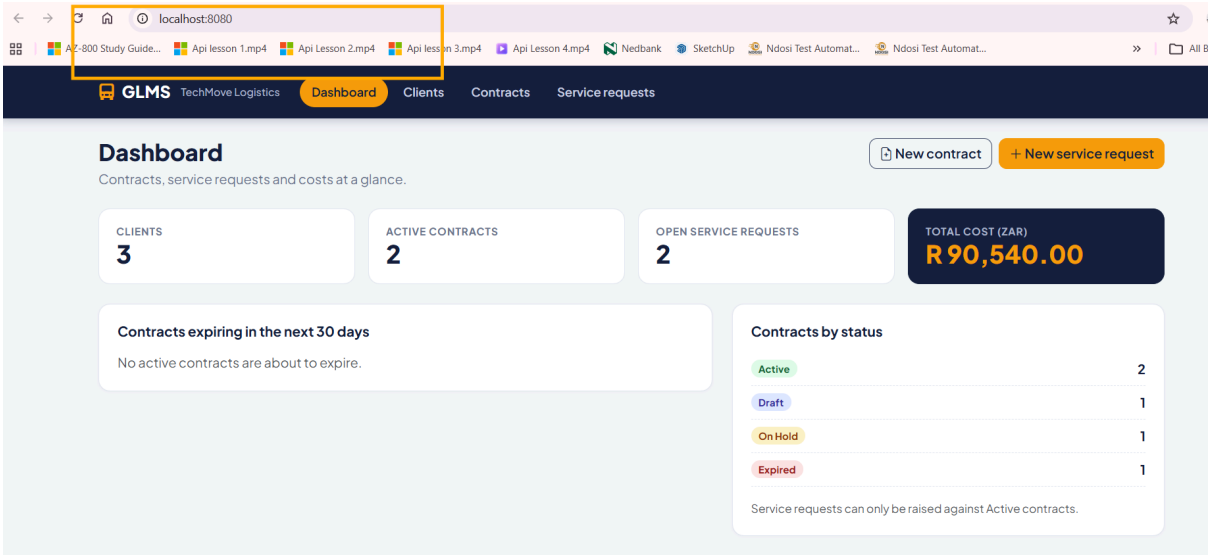


Figure 4: GLMS MVC Frontend Running in Docker

The screenshot demonstrates the ASP.NET Core MVC frontend being accessed through the Docker-published port 8080. The dashboard displays the application's interface and confirms successful browser access to the containerised frontend.

```
PS C:\Users\Andiswa\Documents\Projects\GLMS> Get-Content .\docker-compose.yml

services:

  sql-server-db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: sql-server-db
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_SA_PASSWORD: "${SA_PASSWORD}"
      MSSQL_PID: "Developer"
      MSSQL_MEMORY_LIMIT_MB: "2048"
    volumes:
      - sql_data:/var/opt/mssql
    networks:
      - glms-network
    restart: unless-stopped

  glms-backend-api:
    build:
      context: .
      dockerfile: GLMS.Api/Dockerfile
    container_name: glms-backend-api
    ports:
      - "8081:8080"
    environment:
      ASPNETCORE_ENVIRONMENT: Development
      ASPNETCORE_HTTP_PORTS: "8080"
      ConnectionStrings__GlmsDb: "Server=sql-server-db,1433;Database=GlmsDb;User Id=sa;Password=${SA_PASSWORD};TrustServerCertificate=True;"
    depends_on:
      - sql-server-db
    volumes:
      - agreements:/app/App_Data/SignedAgreements
    networks:
```

```
    networks:
      - glms-network

  glms-frontend-web:
    build:
      context: .
      dockerfile: GLMS.Web/Dockerfile
    container_name: glms-frontend-web
    ports:
      - "8080:8080"
    environment:
      ASPNETCORE_ENVIRONMENT: Development
      ASPNETCORE_HTTP_PORTS: "8080"
      ApiBaseUrl: "http://glms-backend-api:8080/"
      ConnectionStrings__GlmsDb: "Server=sql-server-db,1433;Database=GlmsDb;User Id=sa;Password=${SA_PASSWORD};TrustServerCertificate=True;"
    depends_on:
      - glms-backend-api
    networks:
      - glms-network

volumes:
  sql_data:
  agreements:

networks:
  glms-network:
    driver: bridge

PS C:\Users\Andiswa\Documents\Projects\GLMS>
```

Figure 5: Docker Compose Configuration for the GLMS Architecture

The screenshot illustrates the Docker Compose configuration defining three services: SQL Server, the GLMS Web API and the MVC frontend. The configuration includes container networking, database persistence and port mappings.

Authentication and Security

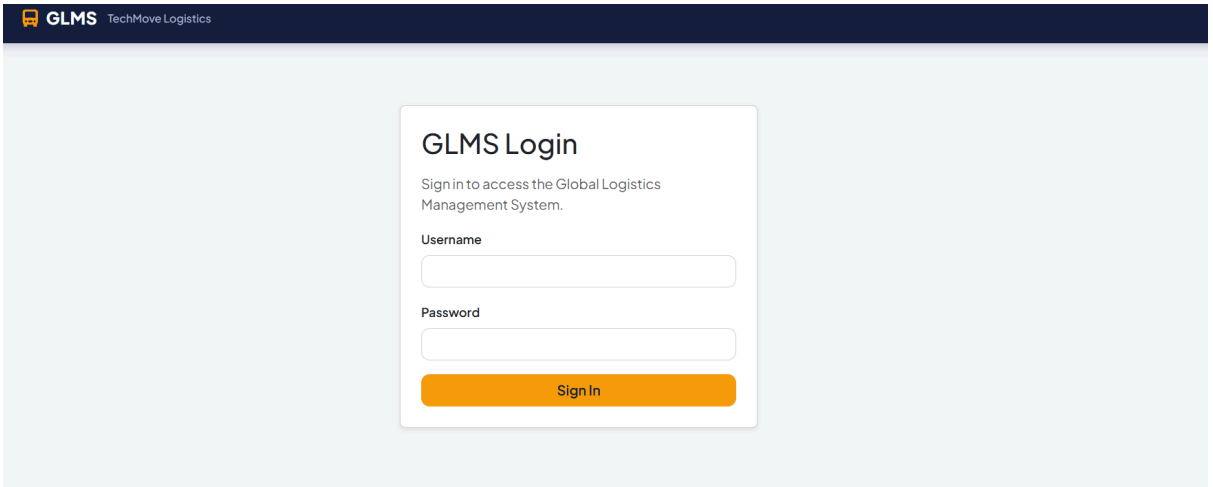


Figure 1: GLMS Administrator Login Interface

The screenshot illustrates the administrator login interface, which requires users to enter valid credentials before accessing the Global Logistics Management System (GLMS).

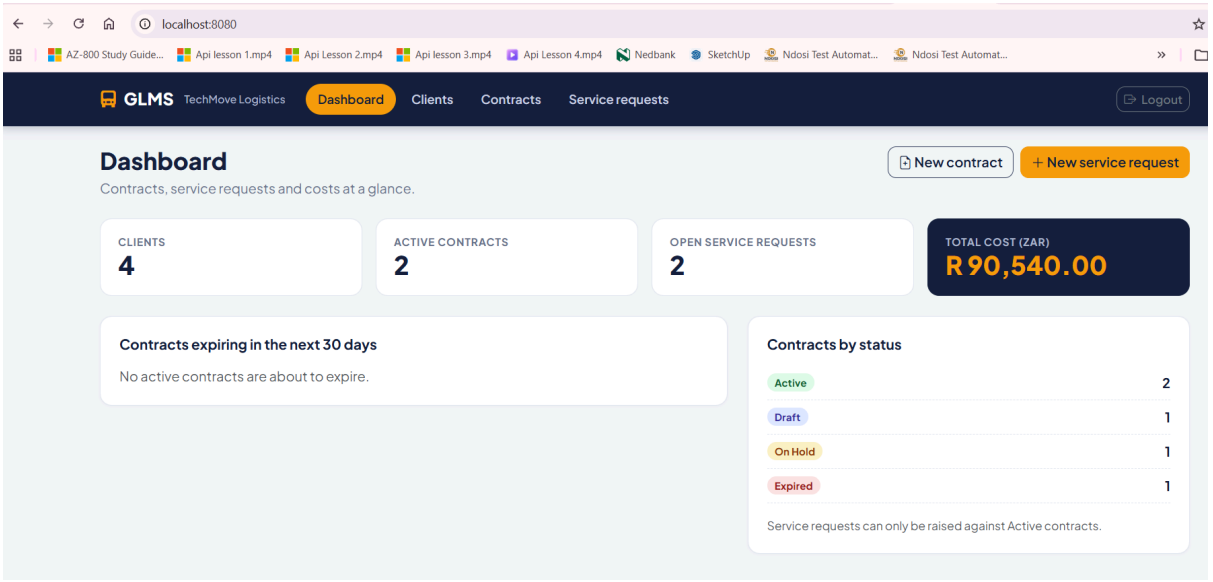


Figure 2: Successful Authentication and Dashboard Access

The screenshot demonstrates successful administrator authentication and access to the GLMS dashboard, including client, contract, and service request information.

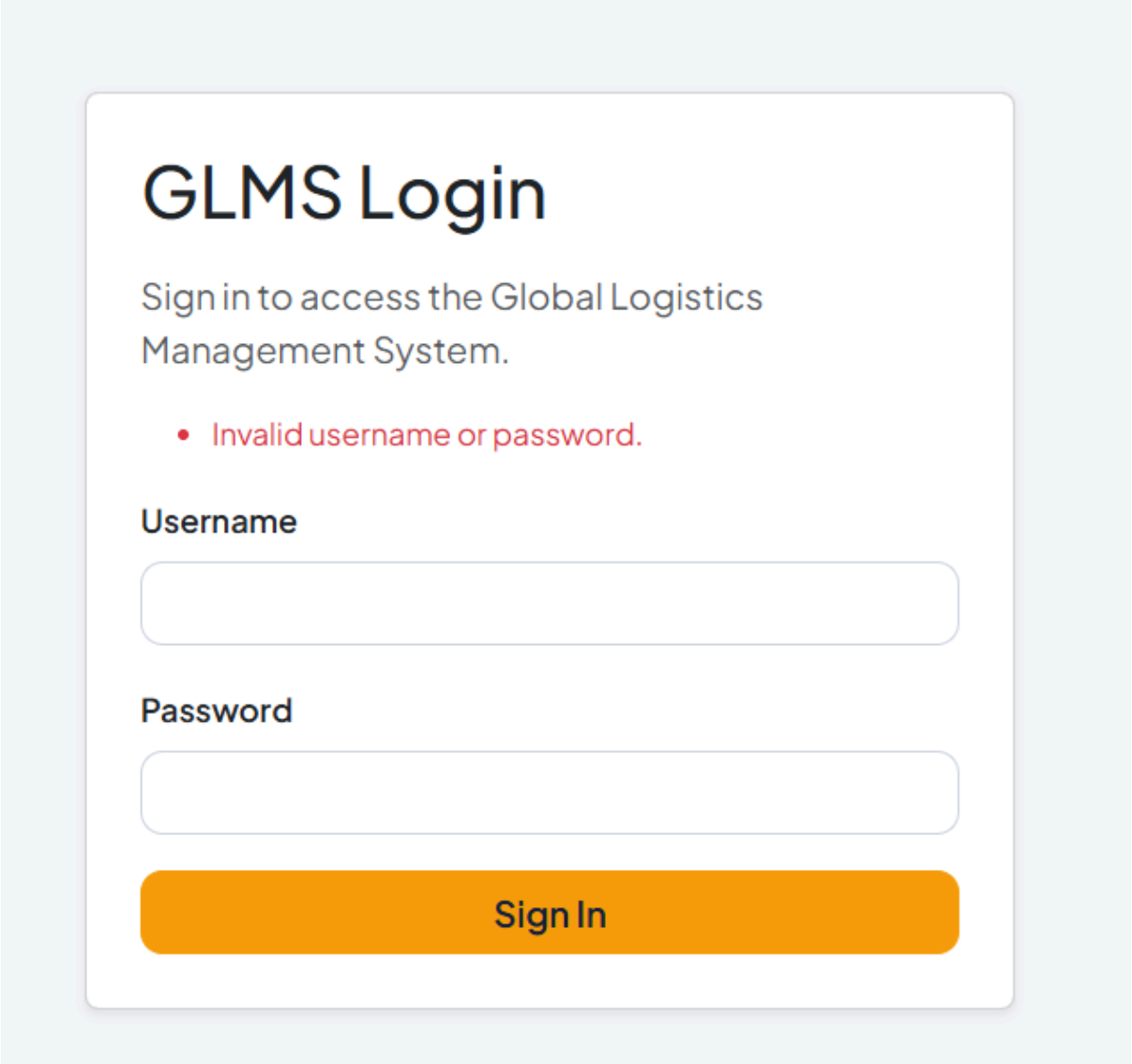


Figure 3: Unauthorized Access Prevention in the MVC Application

The screenshot demonstrates that unauthenticated users attempting to access a protected page are automatically redirected to the login interface, preventing unauthorized access to application resources.



Figure 4: JWT Authentication Enforcement on the Backend API

The screenshot displays the HTTP `401 Unauthorized` response returned when an unauthenticated request is sent directly to the `/api/Clients` endpoint. This confirms that the backend API enforces JWT Bearer authentication independently of the frontend application.

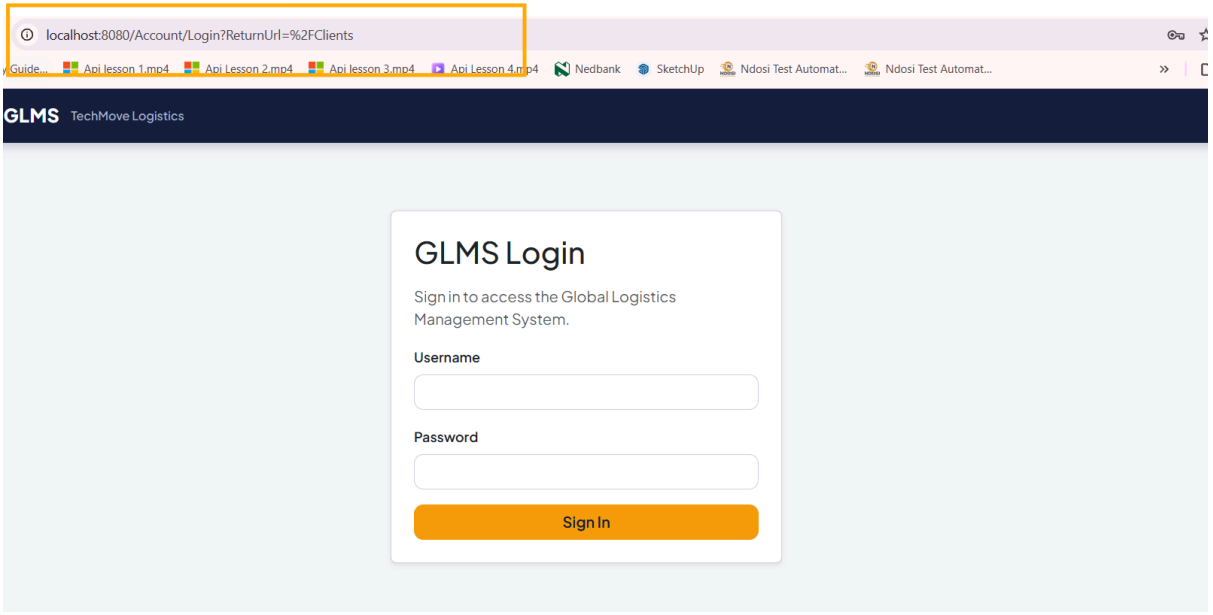


Figure 5: Secure Logout and Session Termination

The screenshot demonstrates that the logout functionality terminates the authenticated session and redirects the user to the login interface. Subsequent attempts to access protected pages require the user to authenticate again.